

Week 2 Day 2: Docker Storage and Network - Volume, Bind Mount, Bridge

Overview

Day 2는 Day 1에서 만든 PostgreSQL container의 데이터가 왜 사라지는지 확인하면서 시작한다. 여기서 volume으로 넘어가되, 하루 전체를 volume 하나로만 쓰지 않는다. container storage와 network를 함께 다룬다. database data를 어디에 둘지, host path를 어떻게 연결할지, container끼리는 어떤 이름으로 통신하는지를 실험한다.

오늘의 핵심 질문은 다음과 같다.

container data와 container network는 container lifecycle과 어떻게 분리되는가?

Day 2는 docker run 기반 storage/network 실험일이다. 각 교시는 CLI 명령을 fenced code block으로 제공하고, 바로 이어서 검증 명령과 cleanup 명령을 둔다. 강의 자료에 들어가는 명령은 사전에 macOS 또는 Linux에서 실행해 실제로 동작하는지 확인한다.

Learning Goals

- Day 1에서 volume 없이 만든 PostgreSQL container를 재생성했을 때 데이터가 사라지는지 확인한다.
- named volume으로 database data lifecycle을 container lifecycle과 분리한다.
- docker volume ls, inspect, rm의 의미와 삭제 위험을 설명한다.
- bind mount와 named volume의 차이를 macOS/Linux host path 관점으로 비교한다.
- custom bridge network에서 container name DNS로 통신한다.
- host port publish와 container 간 network 통신을 구분한다.
- volume+network를 함께 쓰는 PostgreSQL 실험을 완성하고 cleanup한다.

Lesson Index

- 1교시: Day 1 DB container 재기동과 데이터 소실 확인 - volume 없이 만든 PostgreSQL container의 데이터가 남는지 확인
- 2교시: database용 named volume 생성과 연결 - volume을 만들고 PostgreSQL container에 mount한 뒤 데이터를 다시 입력
- 3교시: volume 명령과 cleanup 위험 - volume ls, inspect, rm, dangling volume, bind mount 비교
- 4교시: bind mount와 host path 주의 - macOS/Linux path, read-only mount, host 파일 변경 반영 확인
- 5교시: Docker network 기본 - default bridge, custom bridge, network ls/inspect, attach/detach

- 6교시: container name DNS와 DB client container - host port publish 없이 같은 network에서 PostgreSQL 접속
- 7교시: port publish와 network 차이 - host 접근과 container 간 접근 분리, wrong host/port failure drill
- 8교시: storage/network 통합 실험 - volume+network PostgreSQL, 데이터 보존, DNS 접속, cleanup audit

Practice Files And Assets

자료	용도
assets/day2-storage-network-overview.png	Day 2 storage/network 전체 구조 인포그래픽
Day 1 PostgreSQL evidence	volume 없는 container에서 만든 user/database/table/row 확인 기준
paperclip-pg16 또는 새 PostgreSQL container	데이터 소실/보존 실험 대상
paperclip-pg16-data 같은 named volume	database data persistence 실습 volume
custom Docker network	DB와 client container 간 통신 확인
host path sample folder	bind mount 실험
docker-storage-network-evidence.md	volume/network/bind mount evidence 기록
screenshots/	Docker Desktop volume/container 화면 또는 terminal evidence 파일명 기록
hands-on-lab.md	Day 2 상세 실습 절차를 runtime primitive 중심으로 갱신할 대상

CLI Block Rule

실행할 명령은 prose 안에 흩어 쓰지 않고 아래 형식으로 제공한다.

```
docker volume create paperclip-pg16-data
docker network create paperclip-day2-net
docker run -d --name paperclip-day2-pg \
  --network paperclip-day2-net \
  -e POSTGRES_PASSWORD=postgres \
  -v paperclip-pg16-data:/var/lib/postgresql/data \
  postgres:16
```

검증 명령도 별도 code block으로 둔다.

```
docker ps --filter name=paperclip-day2-pg
docker logs paperclip-day2-pg
```

```
docker run --rm --network paperclip-day2-net postgres:16 \
  pg_isready -h paperclip-day2-pg -U postgres
```

정리 명령도 반드시 함께 제공한다.

```
docker stop paperclip-day2-pg
docker rm paperclip-day2-pg
# data를 삭제해도 되는 실습 volume일 때만 실행
docker volume rm paperclip-pg16-data
docker network rm paperclip-day2-net
```

Linux Preflight Test Evidence

Day 2 volume 실습은 PostgreSQL official image와 Docker named volume을 기준으로 사전 테스트하거나 수업 중 live evidence로 남긴다. 핵심은 출력 문자열 전체가 아니라 container 교체 전후 data가 유지되는지다.

항목	결과
Volume check	no-volume 데이터 소실, named volume 데이터 보존 확인
Bind mount check	host 파일 변경이 container 응답에 반영되는지 확인
Network check	custom network에서 container name으로 pg_isready 또는 SQL 접속 확인
Port/network check	host publish와 container DNS 접속 차이 기록
Cleanup decision	container/network/volume 보존 또는 삭제 위험 확인

Required Evidence

Evidence	제출 기준
No-volume result	Day 1 데이터가 새 container에서 사라졌는지 query 결과로 기록
Volume create result	volume name과 docker volume ls 결과
Mounted container result	container name, image, host port, volume target
SQL reinsert result	user/database/table/insert/query 결과
Container replacement result	새 container가 같은 volume으로 기존 data를 읽는지
Bind mount result	source/destination, read-only 여부, 변경 반영 결과
Network result	custom network name, container name DNS 접속 결과
Cleanup decision	volume keep/remove 결정과 이유
README update	volume/bind-mount/network/port/cleanup/troubleshoot 포함

Extended Hands-on Scope

Day 2의 상세 실습은 [hands-on-lab.md](#)를 storage/network 중심으로 갱신해 진행한다. lesson 본문은 교시별 설명과 핵심 활동을 제공하고, lab guide는 전체 실행 명령, 기대 출력, failure drill, cleanup audit을 제공한다.

Phase	실습 범위	연결 교시
A	Day 1 no-volume DB 데이터 소실 확인	1교시
B	named volume 생성과 PostgreSQL mount	2교시
C	SQL 데이터 재입력	2교시
D	volume ls/inspect/cleanup 위험 확인	3교시
E	bind mount read-only host path 실험	4교시
F	custom network와 container name DNS	5~6교시
G	host port publish와 network 통신 비교	7교시
H	storage/network 통합 failure drill	8교시

Cost And Security Notes

- Day 2는 local Docker와 PostgreSQL official image만 사용한다. cloud database를 만들지 않는다.
- DB password는 실습용이어도 공개 repository에 그대로 올리지 않는다.
- named volume은 container 삭제 후에도 데이터가 남을 수 있으므로 `docker volume rm` 전 데이터를 확인한다.
- bind mount는 host path를 직접 노출하므로 macOS/Linux path와 read/write mode를 확인한다.

Academic And Operational Depth Map

개념	학술/시스템 관점	Docker 실습 연결	운영 질문
Container writable layer	image와 실행 중 변경 상태 분리	no-volume PostgreSQL 재생성	container 삭제 시 어떤 DB 변경이 사라지는가
Named volume	Docker-managed persistent storage	paperclip-pg16-data	container lifecycle과 data lifecycle을 분리했는가
Bind mount	host filesystem을 container namespace에 연결	macOS/Linux path 비교	host path 의존을 배포 환경에 가져가도 되는가
Volume inspect	Docker-managed storage metadata 확인	docker volume inspect	어떤 container가 어떤 data를 쓰는지 추적 가능한가
Custom network	container 간 name-based communication	docker network create, pg_isready -h name	host port 없이 container끼리 통신 가능한가
Port publish	host와 container network boundary	-p host:container	외부 접근과 내부 통신을 구분했는가
Cleanup risk	data deletion boundary	docker volume rm, down -v preview	실습 data와 중요한 data를 구분했는가

End-Of-Day Checklist

- [] Day 1 no-volume PostgreSQL container의 데이터 소실 여부를 확인했다.
- [] database용 named volume을 만들었다.
- [] PostgreSQL container에 named volume을 연결했다.
- [] user/database/table/row를 다시 만들고 query로 확인했다.
- [] container를 교체한 뒤 같은 volume으로 데이터가 살아있는지 확인했다.
- [] docker volume ls와 docker volume inspect 결과를 기록했다.
- [] bind mount로 host 파일 변경 반영과 read-only mode를 확인했다.
- [] custom network에서 container name으로 DB에 접근했다.
- [] host port publish와 container 간 DNS 통신의 차이를 설명했다.
- [] cleanup decision을 기록했다. volume/network/image 삭제 여부를 무심코 결정하지 않았다.
- [] README에 volume/bind-mount/network/port/cleanup/troubleshoot를 작성했다.

Next Connection

Day 3는 image, Dockerfile, layer/cache, tag/digest, registry를 다룬다. Day 4는 env/config, logs, inspect, exec, stats, failure drill을 다룬다. Day 5는 Day 2~4에서 다룬 실행 조건을 여러 아키텍처의 compose.yaml로 옮긴다.